



Kernel density matrices for probabilistic deep learning

Fabio A. González¹ · Raúl Ramos-Pollán² · Joseph Gallego^{1,3}

Received: 23 January 2024 / Accepted: 11 July 2025 / Published online: 7 October 2025
© The Author(s) 2025, corrected publication 2025

Abstract

This paper introduces a novel approach to probabilistic deep learning, kernel density matrices, which provide a simpler yet effective mechanism for representing joint probability distributions of both continuous and discrete random variables. In quantum mechanics, a density matrix is the most general way to describe the state of a quantum system. This work extends the concept of density matrices by allowing them to be defined in a reproducing kernel Hilbert space. This abstraction allows the construction of differentiable models for density estimation, inference, and sampling, and enables their integration into end-to-end deep neural models. In doing so, we provide a versatile representation of marginal and joint probability distributions that allows us to develop a differentiable, compositional, and reversible inference procedure that covers a wide range of machine learning tasks, including density estimation, discriminative learning, and generative modeling. The broad applicability of the framework is illustrated by two examples: an image classification model that can be naturally transformed into a conditional generative model, and a model for learning with label proportions that demonstrates the framework's ability to deal with uncertainty in the training samples. The framework is implemented as a library and is available at: <https://github.com/fagonzalezo/kdm>.

Keywords Quantum machine learning · Density matrix · Kernel methods · Probabilistic deep learning

1 Introduction

The density matrix is a powerful mathematical tool used in quantum mechanics to describe the state of a quantum system (Zwiebach 2022). Unlike the wave function, which provides a complete description of a quantum system in a pure state, the density matrix can describe both pure and mixed states. A mixed state occurs when we have a statistical ensemble (classical uncertainty) of different possible quantum states (quantum uncertainty), rather than a single known state. A density matrix can be thought of as a probability distribution that encodes both the classical and quantum uncertainty of a

quantum system. An attractive feature of the density matrix formalism is that it provides a convenient and powerful computational framework that combines probability and linear algebra. For example, by taking the trace of the product of the density matrix with an operator, one can determine the expectation value of that operator.

Formally, a density matrix is a Hermitian operator that is positive semidefinite, and it acts on a Hilbert space that represents the state space of a quantum system (Nielsen and Chuang 2012). This work introduces an abstraction, the kernel density matrix (KDM), which extends the notion of density matrix to reproducing kernel Hilbert spaces (RKHS). A KDM represents a mixture of kernels that has several interesting features from a machine learning perspective: first, it can represent both discrete and continuous probability distributions; second, it provides efficient kernelized operations for density computation and inference; third, it can be used to develop models for density estimation, inference, and sampling that are differentiable. Accordingly, it could be integrated into end-to-end deep neural models.

The main contributions of this paper are: (i) the extension of the concept of density matrix to RKHSs in the form of KDMs; (ii) demonstrating that KDM is a versatile mecha-

✉ Fabio A. González
fagonzalezo@unal.edu.co

Raúl Ramos-Pollán
raul.ramos@udea.edu.co

Joseph Gallego
jagallegom@unal.edu.co

¹ MindLab, Depto. de Ing. de Sistemas e Industrial,
Universidad Nacional de Colombia, Bogotá, DC, Colombia

² Depto. Ing. de Sistemas, Universidad de Antioquia, Medellín,
Antioquia, Colombia

³ Drexel University, Philadelphia, PA, USA

nism for representing both continuous and discrete probability distributions; (iii) developing a method for conducting inference using KDMs that is differentiable, compositional, and reversible; (iv) presenting parametric and non-parametric learning algorithms for shallow and deep models involving KDMs; and (v) exploring the framework's capabilities in diverse learning tasks.

2 Related work

The model presented in this work can be regarded as a quantum-inspired machine-learning method (Schuld and Petruccione 2018). The majority of quantum machine-learning methods rely on wave functions that depict pure states (Cerezo et al 2023). Density matrices provide an attractive way to represent probability distributions in machine learning, but there have been few studies exploring their potential applications in specific machine learning tasks: classification (Tiwari and Melucci 2019; Sergioli et al 2018; Giuntini et al 2023), clustering (Wolf 2006), and multimodal learning (Li et al 2021) are some examples. The two studies most closely related to the present one are those by Srinivasan et al (2018) and González et al (2022). Srinivasan et al (2018) use a kernel mean map to embed rank-1 density matrices in a RKHS. The authors applied their approach to formulate a hidden quantum Markov model (HQMM) and presented an algorithm based on two-stage regression to learn the parameters of the HQMM. González et al (2022) combine density matrices with random Fourier features (RFF) (Rahimi and Recht 2007) to represent probability distributions, perform inference, and integrate them into deep learning models. The current study presents a more general and efficient formulation than that of González et al (2022), by implicitly defining density matrices in RKHS induced by a kernel, and introducing kernelized versions of operations over these density matrices. In contrast to Srinivasan et al (2018), the framework presented in this study allows the representation of arbitrary rank density matrices and introduces learning algorithms that can be applied to different models including deep end-to-end architectures.

The model proposed in this paper is a kernel-based method that is easily integrable with deep learning architectures due to its formulation. Other authors have explored integrating kernel methods with deep learning. Examples include kernel functions that approximate computations in large, multilayer neural networks (Cho and Saul 2009); neural-kernel conditional mean embedding, which combines deep learning models with the conditional mean embedding strategy to represent conditional probability distributions (Shimizu et al

2024); deep multiple kernel learning methods (Wang et al 2021); and deep Gaussian processes (Ober et al 2021).

Probabilistic deep learning (PDL) refers to the combination of deep neural networks and probabilistic models. PDL can model uncertainty on the input data, the output predictions, and/or the model parameters. Examples of PDL methods include deep Gaussian processes (DGP) (Damianou and Lawrence 2013), Bayesian neural networks (BNN) (Goan and Fookes 2020), generative adversarial networks (GANs) (Goodfellow et al 2020), variational autoencoders (Kingma and Welling 2013), and autoregressive models. The different methods present different approaches to PDL. DGP models the entire training dataset as a stochastic process, while BNN uses a Bayesian approach to model the neural network parameters. Some methods focus on a particular task, such as GANs, which are designed for generation, while others are more versatile and can support a wide range of tasks. Normalizing flows (NF) (Rezende and Mohamed 2015; Papamakarios et al 2021) are an example of the latter and can be applied to tasks such as density estimation, sampling, variational inference, clustering, and classification.

The model presented in this paper can be regarded as a quantum-inspired probabilistic mixture model. Various studies have explored the integration of probabilistic mixture models—particularly Gaussian Mixture Models (GMMs)—into deep neural architectures to capture multi-modal data distributions, model uncertainty, and unify representation learning with clustering. Foundational work on Mixture Density Networks (Bishop 1994) demonstrated how supervised neural networks can output the parameters of a mixture distribution for multi-modal predictions. This approach has been extended in more recent works, including PixelCNN++ (Salimans et al 2017), which employs discretized logistic mixtures for improved pixel likelihood, and WaveNet (van den Oord et al 2016), which applies mixture outputs to generate high-fidelity audio.

For unsupervised modeling, various deep generative and clustering methods now leverage GMM priors to enhance performance. Notable examples include Variational Deep Embedding (Jiang et al 2017), Gaussian Mixture Variational Autoencoders (Dilokthanakul et al 2016) for unsupervised clustering, ClusterGAN (Mukherjee et al 2019) for capturing latent clusters under adversarial training, and Deep Autoencoding Gaussian Mixture Models (Zong et al 2018) for anomaly detection.

Previous works employ GMMs either to output a probability distribution rather than a point estimate in supervised models or as an internal latent representation of data in unsupervised learning. In contrast, the framework presented in this paper applies to a broader range of deep probabilistic learning tasks, including supervised, unsupervised, and

generative learning. Additionally, its foundation in quantum mechanics introduces a novel perspective on probabilistic deep learning, offering unique characteristics and possibilities that merit further investigation.

3 Density matrices and kernel density matrices

3.1 Density matrices

A wave function, denoted as $|\psi\rangle$, is a complex-valued function that lives in a Hilbert space, $\mathcal{H}$, and provides a complete description of the state of a “pure” quantum system. The probability of finding a system in a particular state is given by the square of the absolute value of the projection of the wave function onto the desired state (also called the probability amplitude). This is known as the Born rule (Nielsen and Chuang 2012).

In addition to the quantum uncertainty represented by the wave function, it is possible for the system to have classical uncertainty. In this case, we say that the system is in a “mixed” state, that is, a statistical mixture of N different states, $|\psi_i\rangle$, each with an associated probability p_i , with $\sum_i^N p_i = 1$. This is represented by a density matrix, ρ , which is the more general way to represent the state of a quantum system:

$$\rho = \sum_i^N p_i |\psi_i\rangle \langle \psi_i| \quad (1)$$

where $\langle \psi_i|$ represents the conjugate transpose of $|\psi_i\rangle$. A density matrix is a Hermitian, positive-semidefinite matrix with unit trace, defined on the Hilbert space associated with the quantum system.¹ The Born rule can be extended to compute the probability of finding a system with the state represented by ρ in a state $|\psi\rangle$ after a measurement:

$$p(|\psi\rangle|\rho) = \text{tr}(|\psi\rangle \langle \psi| \rho) = \langle \psi | \rho | \psi \rangle = \sum_i^N p_i |\langle \psi | \psi_i \rangle|^2 \quad (2)$$

Density matrices are a powerful mechanism to represent quantum probability distributions and efficiently perform different calculations: outcomes of quantum measurements, expected values, properties of composite systems, and system dynamics, among others (Nielsen and Chuang 2012).

The following section introduces the concept of a KDM, which can be thought of as a density matrix defined in the RKHS induced by a kernel. In this work, KDMs are used to efficiently represent joint probability distributions and to perform inference, among other things. The defined operations

are differentiable and therefore integrable into deep learning models.

3.2 Kernel density matrices

Definition 1 (Kernel density matrix) A kernel density matrix over a set $\mathbb{X}$ is a triplet $\rho = (\mathbf{C}, \mathbf{p}, k_\theta)$ where $\mathbf{C} = (\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}) \in \mathbb{X}^m$, $\mathbf{p} = (p_1, \dots, p_m) \in \mathbb{R}^m$ and $k_\theta : \mathbb{X} \times \mathbb{X} \rightarrow \mathbb{R}$, such that $\forall \mathbf{x} \in \mathbb{X}$, $k_\theta(\mathbf{x}, \mathbf{x}) = 1$, $\forall i$ $p_i \geq 0$, and $\sum_{i=1}^m p_i = 1$.

The elements of $\mathbf{C}$ are the components of the KDM, and the p_i value represents the mixture weight, or probability, of the component $\mathbf{x}_i$. If $\phi : \mathbb{R}^n \rightarrow \mathcal{H}$ is the mapping to the RKHS $\mathcal{H}$ associated to the kernel k_θ , ρ represents a density matrix defined as in Eq. 1 with components $|\psi_i\rangle = |\phi(\mathbf{x}^{(i)})\rangle$. The projection function associated to a KDM ρ is defined as:

$$f_\rho(\mathbf{x}) = \sum_{\mathbf{x}^{(i)} \in \mathbf{C}} p_i k_\theta^2(\mathbf{x}, \mathbf{x}^{(i)}) \quad (3)$$

If we replace $k_\theta^2(\mathbf{x}, \mathbf{x}^{(i)})$ by $|\langle \phi(\mathbf{x}) | \phi(\mathbf{x}^{(i)}) \rangle|^2$ in Eq. 3, we get the Born rule Eq. 2. In quantum mechanics, it is common to work with complex Hilbert spaces; in this work, we use real-valued kernels over a real domain. This implies that the represented density matrices live in a real Hilbert space. This framework can be easily extended to the complex case, but the real case is sufficient for our goal of representing classical probability distributions.

A central idea in this work is that the projection function in Eq. 3 can be transformed into a probability density function (PDF) in the continuous case, or a probability mass function (PMF) in the discrete case, by multiplying it by a normalization constant that depends on the KDM kernel:

$$\hat{f}_\rho(\mathbf{x}) = \mathcal{M}_k f_\rho(\mathbf{x}) \quad (4)$$

3.2.1 Discrete kernel density matrices

A discrete KDM refers to a KDM in which the corresponding kernel is associated with an RKHS of finite dimension. In this work, we will use $\mathbb{X} = \mathbb{R}^n$ along with the cosine kernel ($k_{\cos}(\mathbf{x}, \mathbf{y}) = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\sqrt{\langle \mathbf{x}, \mathbf{x} \rangle \langle \mathbf{y}, \mathbf{y} \rangle}}$), which corresponds to a dot product between normalized vectors, to represent discrete probability distributions. The normalization constant for this kernel is $\mathcal{M}_{k_{\cos}} = 1$. It is possible to use any kernel that satisfies $\forall \mathbf{x} \in \mathbb{X}$, $k(\mathbf{x}, \mathbf{x}) = 1$ and has an associated RKHS of finite dimension.

The following proposition shows that a KDM $\rho_{\mathbf{x}} = (\mathbf{C}, \mathbf{p}, k_{\cos})$ represents a probability distribution² for a random variable $\mathbf{x}$ that can take a finite set of values.

¹ If the Hilbert space is infinite-dimensional, we use the term “density operators” instead of “density matrices.”

² We will use the notation $\rho_{\mathbf{x}}$ to indicate that the KDM ρ represents a probability distribution for the random variable $\mathbf{x}$.

Proposition 1 Let $\rho_{\mathbf{x}} = (\mathbf{C}, \mathbf{p}, k_{\cos})$ be a KDM over $\mathbb{R}^n$; let $\mathbb{X} = \{\mathbf{b}^{(1)}, \dots, \mathbf{b}^{(n)}\} \subset \mathbb{R}^n$ be an orthogonal basis of $\mathbb{R}^n$, then $\{f_{\rho}(\mathbf{b}^{(i)})\}_{i=1,\dots,n}$ is a categorical probability distribution for the random variable $\mathbf{x} \in \mathbb{X}$.

In particular, if we use the canonical basis $\mathbb{X} = \{\mathbf{e}^{(1)}, \dots, \mathbf{e}^{(n)}\}$, the function in Eq. 4, defines a valid categorical PMF over $\mathbb{X}$:

$$\hat{f}_{\rho_{\mathbf{x}}}(\mathbf{x}) = f_{\rho_{\mathbf{x}}}(\mathbf{x}), \text{ for } \mathbf{x} \in \{\mathbf{e}^{(i)}\}_{i=1,\dots,n} \quad (5)$$

As an example, consider the discrete probability distribution $\mathbf{p} = (0.2, 0.3, 0.5)$. It can be represented by the KDM $\rho_0 = (((1, 0, 0), (0, 1, 0), (0, 0, 1)), (0.2, 0.3, 0.5), k_{\cos})$, also, it can be represented by the KDM $\rho_1 = (((\sqrt{0.2}, \sqrt{0.3}, \sqrt{0.5}), (1), k_{\cos})$.

3.2.2 Continuous kernel density matrices

A continuous KDM is a KDM over $\mathbb{R}^n$ with a radial basis kernel $k(x, y) = K(x - y)$ that satisfies:

$$\begin{aligned} K(0) &= 1 \\ \int K^2(x) dx &\leq \infty \\ \int x K^2(x) dx &= 0 \\ \int x^2 K^2(x) dx &> 0 \end{aligned} \quad (6)$$

There are different kernels that satisfy the conditions in Eq. 6, including the Gaussian, Epanechnikov, and tricube kernels (Wasserman et al 2006). In this work, we will use the Gaussian (or RBF) kernel, $k_{\text{rbf},\sigma}(\mathbf{x}, \mathbf{y}) = e^{-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{2\sigma^2}}$.

A KDM $\rho_{\mathbf{x}} = (\mathbf{C}, \mathbf{p}, k_{\text{rbf}})$ with $\mathbf{C} \subseteq \mathbb{R}^n$ represents a continuous probability distribution of a random variable $\mathbf{x} \in \mathbb{R}^n$ with a PDF given by Eq. 4 with normalizing constant $\mathcal{M}_{k_{\text{rbf}}} = \frac{1}{\sqrt{(2\pi)^n}(\sqrt{2}\sigma)^n}$. It is not difficult to check that the PDF corresponding to $\rho_{\mathbf{x}}$, $\hat{f}_{\rho_{\mathbf{x}}}$, is a valid PDF over $\mathbb{R}^n$. In fact, Eq. 4, with normalizing constant $\mathcal{M}_{k_{\text{rbf}}}$, is a generalization of the non-parametric estimation approach to density estimation with kernels known as kernel density estimation (KDE) (Rosenblatt 1956; Parzen 1962). As shown in the present work, KDMs can be used in a wider range of applications and can be learned using non-parametric and parametric approaches.

3.3 Density estimation with kernel density matrices

Since a KDM extends the notion of a kernel density estimator, it can be used for non-parametric density estimation—that is, by using all training points to define the KDM. Specifically,

in the case of a KDM with an RBF kernel, Eq. 4 corresponds to a KDE density estimator with a Gaussian kernel and a bandwidth parameter of $\frac{\sigma}{\sqrt{2}}$. The following theorem, due to Parzen, guarantees that the estimator in Eq. 4 converges in probability to the true PDF:

Theorem 2 (Parzen (1962)) Let $\mathbf{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(\ell)}\} \subseteq \mathbb{R}^n$ be a set of iid samples drawn from a probability distribution with PDF f ; let $\rho_{\mathbf{x}}^{\ell} = (\mathbf{D}, \mathbf{p} = (\frac{1}{\ell}, \dots, \frac{1}{\ell}), k_{\text{rbf},\sigma_{\ell}})$ be a KDM; and let $\hat{f}_{\rho_{\mathbf{x}}^{\ell}}$ be defined as in Eq. 4. Assume that f is continuous at $\mathbf{x}$ and that $\sigma_{\ell} \rightarrow 0$ and $\ell\sigma_{\ell} \rightarrow \infty$ as $\ell \rightarrow \infty$. Then, $\hat{f}_{\rho_{\mathbf{x}}^{\ell}}(\mathbf{x}) \xrightarrow{P} f(\mathbf{x})$.

The following proposition corresponds to the analogous result for categorical probability distributions.

Proposition 3 Let $\mathbb{X} = \{\mathbf{e}^{(1)}, \dots, \mathbf{e}^{(n)}\}$ be the canonical basis of $\mathbb{R}^n$; let $\mathbf{D} = (x_1, \dots, x_{\ell}) \in (\mathbb{R}^n)^{\ell}$ be a set of iid samples drawn from $\mathbb{X}$ with categorical probability distribution $\mathbf{p} = (p_1, \dots, p_n)$; let $\rho_{\mathbf{x}}^{\ell} = (\mathbf{D}, \mathbf{p} = (\frac{1}{\ell}, \dots, \frac{1}{\ell}), k_{\cos})$ be a KDM; and let $\hat{f}_{\rho_{\mathbf{x}}^{\ell}}(i)$ be defined as in Eq. 5. Then, $\hat{f}_{\rho_{\mathbf{x}}^{\ell}}(i) \rightarrow p_i$ as $\ell \rightarrow \infty$.

The previous results are asymptotic. For a finite-sample-size Parzen window estimator, Zhang et al (2005) derived an error bound. However, the Parzen window method requires the entire training dataset during prediction, leading to significant computational and memory costs. A more efficient approach is discussed next.

It is also possible to perform parametric density estimation by maximizing the likelihood of the KDM parameters given a training dataset. Given a KDM with K components $\rho = (\mathbf{C} = (\mathbf{c}^{(1)}, \dots, \mathbf{c}^{(K)}), \mathbf{p} = (p_1, \dots, p_K), k_{\theta})$, where θ indicates a set of parameters of the kernel, and a training data set $\mathbf{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_{\ell}\}$, solving the following optimization problem:

$$\max_{\mathbf{C}, \mathbf{p}, \theta} \sum_{i=1}^{\ell} \log \hat{f}_{\rho}(x_i) \quad (7)$$

will find a KDM that maximizes the probability density of the samples in $\mathbf{D}$. For the cosine or RBF kernels this is equivalent to perform maximum likelihood estimation.³

3.4 Joint densities with kernel density matrices

In quantum mechanics, the state of a bipartite system with subsystems A and B is represented by a density matrix in the

³ The optimization problem in Eq. 7 could be ill-posed, e.g., for the RBF kernel, Eq. 7 could get arbitrary large values if one sample x_i coincides with one of the KDM components and the σ parameter of the kernel goes to zero. This could be dealt with using techniques such as constrained optimization or regularization.

tensor product Hilbert space $\mathcal{H}_A \otimes \mathcal{H}_B$, where $\mathcal{H}_A$ and $\mathcal{H}_B$ are the representation spaces of A and B respectively (Nielsen and Chuang 2012). Following the same line of thought, a joint KDM for random variables $\mathbf{x} \in \mathbb{X}$ and $\mathbf{y} \in \mathbb{Y}$ is defined as follows: $\rho_{\mathbf{x}, \mathbf{y}} = (\mathbf{C}, \mathbf{p}, k_{\mathbb{X}} \otimes k_{\mathbb{Y}})$ where $\mathbf{C} \subseteq \mathbb{X} \times \mathbb{Y}$, $k_{\mathbb{X}}$ and $k_{\mathbb{Y}}$ are kernels over $\mathbb{X}$ and $\mathbb{Y}$ respectively, and $k_{\mathbb{X}} \otimes k_{\mathbb{Y}}((\mathbf{x}, \mathbf{y}), (\mathbf{x}', \mathbf{y}')) = k_{\mathbb{X}}(\mathbf{x}, \mathbf{x}')k_{\mathbb{Y}}(\mathbf{y}, \mathbf{y}')$. If $\mathcal{M}_{k_{\mathbb{X}}}$ and $\mathcal{M}_{k_{\mathbb{Y}}}$ are the normalizing constants corresponding to the kernels $k_{\mathbb{X}}$ and $k_{\mathbb{Y}}$ respectively, the normalizing constant for $k_{\mathbb{X}} \otimes k_{\mathbb{Y}}$ in Eq. 4 is $\mathcal{M}_{k_{\mathbb{X}} \otimes k_{\mathbb{Y}}} = \mathcal{M}_{k_{\mathbb{X}}} \mathcal{M}_{k_{\mathbb{Y}}}$. It is also possible to easily calculate a marginal KDM from a joint KDM. Suppose that $k_{\mathbb{X}} = k_{\text{rbf}}$, then a marginal KDM $\rho_{\mathbf{y}}$ must satisfy:

$$\begin{aligned} \hat{f}_{\rho_{\mathbf{y}}}(\mathbf{y}) &= \int \hat{f}_{\rho_{\mathbf{x}, \mathbf{y}}}(\mathbf{x}, \mathbf{y}) d\mathbf{x} \\ &= \int \sum_i p_i \mathcal{M}_{k_{\mathbb{X}}} \mathcal{M}_{k_{\mathbb{Y}}} k_{\mathbb{X}}^2(\mathbf{x}^{(i)}, \mathbf{x}) k_{\mathbb{Y}}^2(\mathbf{y}^{(i)}, \mathbf{y}) d\mathbf{x} \quad (8) \\ &= \sum_i p_i \mathcal{M}_{k_{\mathbb{Y}}} k_{\mathbb{Y}}^2(\mathbf{y}^{(i)}, \mathbf{y}) \end{aligned}$$

It is not difficult to see that $\rho_{\mathbf{y}} = (\{\mathbf{y}^{(i)} | (\mathbf{x}^{(i)}, \mathbf{y}^{(i)}) \in \mathbf{C}\}, \mathbf{p}, k_{\mathbb{Y}})$ satisfies Eq. 8. The same argument applies for defining $\rho_{\mathbf{x}}$. For $k_{\mathbb{X}} = k_{\text{cos}}$, we replace the integral in Eq. 8 by a sum over a basis (see Proposition 1) to obtain an analogous result.

3.5 Inference with kernel density matrices

Inference aims to estimate unknown output variables based on known input variables and a model's parameters. A probabilistic approach models the input–output relationship as a probability distribution, such as $p(\mathbf{x}', \mathbf{y}')$, which captures uncertainty arising from the training data-generation process. Additional uncertainty may stem from imprecise input data during prediction, which can be represented by a probability distribution over the input domain, $p(\mathbf{x})$.

To distinguish between training and prediction phases, we use different random variables: $\mathbf{x}'$ denotes an input sample from the training data, while $\mathbf{x}$ represents a new input provided to the trained model during inference. Importantly, the input distribution $p(\mathbf{x})$ does not necessarily correspond to the marginalized training distribution:

$$p(\mathbf{x}) \neq \int p(\mathbf{x}', \mathbf{y}' = \mathbf{y}) d\mathbf{y}.$$

When predicting output variables, both sources of uncertainty—the inherent variability in data and the uncertainty in the input values—must be considered. These uncertainties are reflected in the output distribution, $p(\mathbf{y})$. Inference, therefore, transforms the probability distribution of input variables, $p(\mathbf{x})$, into a corresponding distribution of output

variables, $p(\mathbf{y})$, using the joint distribution of inputs and outputs, $p(\mathbf{x}', \mathbf{y}')$.

We will demonstrate how this process can be modeled using KDMs. Each one of the probability distributions $p(\mathbf{x})$, $p(\mathbf{x}', \mathbf{y}')$ and $p(\mathbf{y})$ are represented by KDMs as follows:

$$\rho_{\mathbf{x}} = ((\mathbf{x}^{(i)})_{i=1\dots m}, (p_i)_{i=1\dots m}, k_{\mathbb{X}}) \quad (9)$$

$$\rho_{\mathbf{x}', \mathbf{y}'} = (((\mathbf{x}'^{(i)}, \mathbf{y}'^{(i)}))_{i=1\dots m'}, (p'_i)_{i=1\dots m'}, k_{\mathbb{X}} \otimes k_{\mathbb{Y}}) \quad (10)$$

$$\rho_{\mathbf{y}} = ((\mathbf{y}'^{(i)})_{i=1\dots m'}, (p''_i)_{i=1\dots m'}, k_{\mathbb{Y}}) \quad (11)$$

Notice that if we have a unique input sample $\mathbf{x}$, $\rho_{\mathbf{x}}$ would have only one component. In general, we have a probability distribution represented by the KDM $\rho_{\mathbf{x}}$ with m components. Also, note that $k_{\mathbb{X}}$ in $\rho_{\mathbf{x}}$, is the same kernel as in $\rho_{\mathbf{x}', \mathbf{y}'}$. Likewise, $k_{\mathbb{Y}}$ in $\rho_{\mathbf{y}}$, is the same kernel as in $\rho_{\mathbf{x}', \mathbf{y}'}$. Also, the $\mathbf{y}'^{(i)}$ components of the KDM $\rho_{\mathbf{y}}$ are same as the ones of the KDM $\rho_{\mathbf{x}', \mathbf{y}'}$. The probabilities of the inferred $\rho_{\mathbf{y}}$ KDM are given by the following expression:

$$p''_i = \sum_{\ell=1}^m \frac{p_{\ell} p'_i (k_{\mathbb{X}}(\mathbf{x}^{(\ell)}, \mathbf{x}'^{(i)}))^2}{\sum_{j=1}^{m'} p'_j (k_{\mathbb{X}}(\mathbf{x}^{(\ell)}, \mathbf{x}'^{(j)}))^2}, \text{ for } i = 1 \dots m' \quad (12)$$

Algorithm 1 Inference with kernel density matrices.

input: Input KDM $\rho_{\mathbf{x}} = ((\mathbf{x}^{(i)})_{i=1\dots m}, (p_i)_{i=1\dots m}, k_{\mathbb{X}})$, joint KDM $\rho_{\mathbf{x}', \mathbf{y}'} = (((\mathbf{x}'^{(i)}, \mathbf{y}'^{(i)}))_{i=1\dots m'}, (p'_i)_{i=1\dots m'}, k_{\mathbb{X}} \otimes k_{\mathbb{Y}})$

- 1: $\mathbf{C}_{\mathbf{y}} \leftarrow \{\mathbf{y}'^{(i)}\}_{i=1\dots m'}$
 - 2: $\mathbf{p}_{\mathbf{y}} \leftarrow (p''_k)_{k=1\dots m'}$ where p''_k is calculated using Eq. 12
 - 3: **return** $\rho_{\mathbf{y}} = (\mathbf{C}_{\mathbf{y}}, \mathbf{p}_{\mathbf{y}}, k_{\mathbb{Y}})$
-

The time complexity of the inference process (Algorithm 1) based on Eq. 12 is $O(mm'n)$ where n is the dimension of the input space. This inference procedure is similar to Nadaraya-Watson kernel regression (NWKR) (Nadaraya 1964; Watson 1964), which uses a kernel function to assign local weights to each output training sample according to the input sample's similarity with each input training sample. However, NWKR only produces a point estimate of the output variable, while we obtain a full probability distribution represented as a KDM.

In addition to its expressiveness and flexibility, KDM-based inference has two advantages:

- **Compositionality.** Given two joint KDMs $\rho_{\mathbf{x}', \mathbf{y}'}$ and $\rho_{\mathbf{y}', \mathbf{z}'}$ and input KDM $\rho_{\mathbf{x}}$, we can infer $\rho_{\mathbf{y}}$ from $(\rho_{\mathbf{x}', \mathbf{y}'}, \rho_{\mathbf{x}})$ and then infer $\rho_{\mathbf{z}}$ from $(\rho_{\mathbf{y}', \mathbf{z}'}, \rho_{\mathbf{y}})$.
- **Reversibility.** Thanks to the symmetry of the joint KDMs $\rho_{\mathbf{x}', \mathbf{y}'}$, it can be used to infer $\rho_{\mathbf{x}}$ from $\rho_{\mathbf{y}}$ as well as to infer $\rho_{\mathbf{y}}$ from $\rho_{\mathbf{x}}$.

From the point of view of quantum mechanics, $\rho_{\mathbf{x}', \mathbf{y}'}$ is a density matrix representing the state of a bipartite system, and the inference process corresponds to a measurement operation that collapses the $\mathbf{x}$ subsystem to a state $\rho_{\mathbf{x}}$. This collapse affects the $\mathbf{y}$ subsystem whose state is changed to be $\rho_{\mathbf{y}}$. With density matrices, this process corresponds to applying a collapse operator to $\rho_{\mathbf{x}', \mathbf{y}'}$ and computing a partial trace (González et al 2022). With our KDM representation, this process is efficiently done by Eq. 12.

The following proposition shows that the probabilities assigned by Eq. 12 generate a KDM that is equivalent to the one calculated by Bayesian inference.

Proposition 4 *Let $\mathbf{x}'$ and $\mathbf{y}'$ be random variables with a joint probability distribution represented by a KDM $\rho_{\mathbf{x}', \mathbf{y}'}$ (Eq. 10), and let $\rho_{\mathbf{x}}$ be the KDM defined by Eq. 9), then the KDM $\rho_{\mathbf{y}}$ (Eqs. 11 and 12) represents a predicted probability distribution with PDF (or PMF):*

$$\hat{f}_{\rho_{\mathbf{y}}}(\mathbf{y}) = \sum_{\ell=1}^m p_{\ell} \hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{y} | \mathbf{x}^{(\ell)}) \quad (13)$$

where

$$\begin{aligned} \bullet \quad \hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{y} | \mathbf{x}^{(\ell)}) &= \frac{\hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{x}^{(\ell)}, \mathbf{y})}{\int \hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{x}^{(\ell)}, \mathbf{y}) d\mathbf{y}} \text{ for } k_{\mathbb{Y}} = k_{rbf} \\ \bullet \quad \hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{y} | \mathbf{x}^{(\ell)}) &= \frac{\hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{x}^{(\ell)}, \mathbf{y})}{\sum_{i=1}^n \hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{x}^{(\ell)}, \mathbf{e}^{(i)})} \text{ for } k_{\mathbb{Y}} = k_{cos} \end{aligned}$$

The parameters of the inference model correspond to the parameters of the KDM $\rho_{\mathbf{x}', \mathbf{y}'}$. These parameters can be estimated in three different ways:

- Non-parametric. Use the training dataset as the parameters of $\rho_{\mathbf{x}', \mathbf{y}'}$ as described in Sect. 3.3. This leads to a memory-based learning method that may not scale well to large training datasets.
- Maximum likelihood learning. Estimate the parameters of $\rho_{\mathbf{x}', \mathbf{y}'}$ by maximizing the probability density assigned by $\rho_{\mathbf{x}', \mathbf{y}'}$ to the training dataset (Eq. 7) (Algorithm 2).
- Discriminative learning. Use Eq. 12 to perform a forward pass of an input KDM $\rho_{\mathbf{x}}$ that can represent an individual sample (no uncertainty) or a distribution over the input space (uncertainty). Then, we minimize a suitable loss function (such as cross-entropy for classification or mean squared error for regression) with respect to the model parameters using gradient descent (Algorithm 3).

For discriminative learning, the loss function depends on the kind of output variable, which is determined by the kernel $k_{\mathbb{Y}}$. Given an output KDM $\rho_{\mathbf{y}} = ((\mathbf{y}^{(i)})_{i=1 \dots m'}, (p_i'')_{i=1 \dots m'}$,

Algorithm 2 Maximum likelihood training.

input: training dataset $\mathbf{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1 \dots \ell}$, kernels $k_{\mathbb{X}}$ and $k_{\mathbb{Y}}$, number of components m'

- 1: Initialize $\mathbf{C}_{\mathbf{x}' \mathbf{y}'} = ((\mathbf{x}'^{(i)}, \mathbf{y}'^{(i)}))_{i=1 \dots m'}$ with a random sample from $\mathbf{D}$
- 2: Initialize $\mathbf{p}_{\mathbf{x}' \mathbf{y}'} \leftarrow (\frac{1}{m'})_{i=1 \dots m'}$
- 3: $\rho_{\mathbf{x}', \mathbf{y}'} = (\mathbf{C}_{\mathbf{x}' \mathbf{y}'}, \mathbf{p}_{\mathbf{x}' \mathbf{y}'}, k_{\mathbb{X}} \otimes k_{\mathbb{Y}})$
- 4: Use gradient-based optimization to find:

$$\max_{\mathbf{C}_{\mathbf{x}' \mathbf{y}'}, \mathbf{p}_{\mathbf{x}' \mathbf{y}'}, \theta} \sum_{i=1}^{\ell} \log \hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$$

5: **return** $\rho_{\mathbf{x}', \mathbf{y}'} = (\mathbf{C}_{\mathbf{x}' \mathbf{y}'}, \mathbf{p}_{\mathbf{x}' \mathbf{y}'}, k_{\mathbb{X}} \otimes k_{\mathbb{Y}})$

Algorithm 3 Discriminative training.

input: training dataset $\mathbf{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1 \dots \ell}$, kernels $k_{\mathbb{X}, \sigma}$ and $k_{\mathbb{Y}}$, number of components m' , loss function $\mathcal{L}$

- 1: Initialize $\mathbf{C}_{\mathbf{x}' \mathbf{y}'} \leftarrow ((\mathbf{x}'^{(i)}, \mathbf{y}'^{(i)}))_{i=1 \dots m'}$ with a random sample from $\mathbf{D}$
- 2: $\mathbf{p}_{\mathbf{x}' \mathbf{y}'} \leftarrow (\frac{1}{m'})_{i=1 \dots m'}$
- 3: $\rho_{\mathbf{x}', \mathbf{y}'} \leftarrow (\mathbf{C}_{\mathbf{x}' \mathbf{y}'}, \mathbf{p}_{\mathbf{x}' \mathbf{y}'}, k_{\mathbb{X}, \sigma} \otimes k_{\mathbb{Y}})$
- 4: $\rho_{\mathbf{x}}^{(i)} \leftarrow ((\mathbf{x}^{(i)}, (1), k_{\mathbb{X}, \sigma}))$ for all $i = 1 \dots \ell$ ▷ create a KDM for each input sample
- 5: **repeat**
- 6: Calculate $\rho_{\mathbf{y}}^{(i)}$ using Eq. 12 with parameters $\rho^{(i)}(\mathbf{x})$ and $\rho(\mathbf{x}', \mathbf{y}')$ for all $i = 1 \dots \ell$
- 7: Perform a step of gradient-based method to optimize: $\min_{\mathbf{C}_{\mathbf{x}' \mathbf{y}'}, \mathbf{p}_{\mathbf{x}' \mathbf{y}'}, \sigma} \sum_{i=1}^n \mathcal{L}(\rho_{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)})$
- 8: **until** stop criteria is met
- 9: **return** $\rho_{\mathbf{x}', \mathbf{y}'} = (\mathbf{C}_{\mathbf{x}' \mathbf{y}'}, \mathbf{p}_{\mathbf{x}' \mathbf{y}'}, k_{\mathbb{X}} \otimes k_{\mathbb{Y}})$

$k_{\mathbb{Y}}$), where $k_{\mathbb{Y}} = k_{cos}$, and a real output label $\mathbf{y} \in \mathbb{R}^n$, with $\sum_{j=1}^n y_j = 1$ we can use, for instance, a cross-entropy loss:

$$\mathcal{L}_{xe}(\rho_{\mathbf{y}}, \mathbf{y}) = \text{CrossEntropy}(\boldsymbol{\pi}, \mathbf{y}), \quad (14)$$

where

$$\boldsymbol{\pi}_j = \sum_{i=1}^{m'} p_i'' (\mathbf{y}_j^{(i)})^2, \text{ for } j = 1 \dots n \quad (15)$$

When $k_{\mathbb{Y}} = k_{rbf}$, we have a regression problem; in this case, we can use a loss function such as mean square error:

$$\mathcal{L}_{mse}(\rho_{\mathbf{y}}, \mathbf{y}^{(i)}) = \text{mse}(\hat{\mathbf{y}}, \mathbf{y}^{(i)}), \quad (16)$$

where $\hat{\mathbf{y}} = \sum_{j=1}^{m'} p_j'' \mathbf{y}_j^{(i)}$.

Maximum likelihood learning can be implemented in different ways. One approach is to use stochastic gradient ascent to maximize the log-likelihood (Eq. 7). However, to prevent the optimization process from reaching a trivial solution by reducing the Gaussian kernel's bandwidth (σ), additional measures such as regularization (e.g., penalizing the size

of $\frac{1}{\sigma}$ or using an inverse Gamma penalization for σ^2) or hard thresholding are necessary. This, in turn, requires hyperparameter tuning to determine an appropriate threshold or regularization parameter.

Alternatively, maximum likelihood learning can be approached using expectation maximization (EM) (Dempster et al 1977). EM is commonly employed for training probabilistic mixture models via maximum likelihood, making its adaptation to KDM-based models relatively straightforward. A key advantage of EM is its convergence guarantees. However, its direct application presents some challenges. First, the expectation step requires computing statistics over all training points, which can be computationally expensive for large datasets. In contrast, gradient-based methods efficiently handle large datasets via stochastic approximations, such as mini-batch training (Cappé and Moulines 2009). Second, integrating the KDM framework with deep learning is more naturally achieved using gradient-based optimization. While EM can be adapted for deep neural networks (Greff et al 2017), this lies beyond the scope of the present work but could be a promising direction for future research.

Kernel mixture models struggle with high-dimensional data. In classic methods such as Manifold Parzen Windows (MPWs) (Vincent and Bengio 2002), this issue is addressed by exploiting the fact that input data often lies on a lower-dimensional manifold. The local structure of this manifold is captured by assigning each training data point a covariance matrix, which is adjusted using local (neighboring) data or a combination of local and global data (Bengio et al 2005). However, this approach becomes prohibitively expensive even for medium-sized training datasets.

The KDM framework addresses this more efficiently and at scale by coupling inference modules with a deep neural network (encoder) that reduces input dimensionality and is trained end-to-end. While this approach works well for discriminative learning, in maximum likelihood learning, the encoder might collapse all samples to a single high-likelihood point, resulting in a trivial model. A potential solution is to pretrain the encoder with discriminative learning, then freeze it before switching to maximum likelihood learning.

Additionally, in cases where the output is high-dimensional—such as supervised problems with thousands of classes—the number of components required for the joint KDM model increases significantly. Specifically, for a joint KDM, modeling the joint probability of inputs and outputs, the model would need nk components, where n represents the number of components per class and k is the number of classes.

3.6 Sampling from kernel density matrices

Sampling from a KDM $\rho_{\mathbf{x}} = ((\mathbf{x}^{(i)})_{i=1\dots m}, (p_i)_{i=1\dots m}, k_{\text{rbf},\sigma})$ with $\mathbf{x}^{(i)} \in \mathbb{R}^n$ can be performed in two steps. First,

draw a sample i' from a categorical distribution with probabilities $(p_i)_{i=1\dots m}$. Second, draw a sample from a normal distribution $\mathcal{N}(\mathbf{x}^{(i')}, \frac{\sigma^2}{2} \mathbf{I}_n)$, where $\mathbf{I}_n$ is the n -dimensional identity matrix.

To sample from a KDM $\rho_{\mathbf{x}} = ((\mathbf{x}^{(i)})_{i=1\dots m}, (p_i)_{i=1\dots m}, k_{\text{cos}})$, we draw a sample from a categorical distribution with probabilities given by $(\pi_j)_{j=1\dots n}$, where π_j is defined in the same way as in Eq. 15.

Sampling layers that sample from a KDM can also be included in a deep model. To maintain differentiability, we can use the Gumbel-softmax reparameterization for sampling categorical variables, as proposed by Jang et al (2017), along with the well-known reparameterization trick for Gaussian distributions. Another alternative is to use implicit reparameterization gradients, Figurnov et al (2018).

4 Experiments

The purpose of this section is to demonstrate the versatility and flexibility of the framework with two different learning tasks. Rather than focusing on improving the performance of state-of-the-art methods, these experiments illustrate the unique features of the framework and the novel perspective it offers for PDL modeling. First, we present an example of a classification model based on the framework, highlighting the reversibility of KDM-based inference, which enables the model to function as both a classification and a conditional generative model. In the second example, we address the problem of learning from label proportions, where individual training sample labels are unavailable, and only the proportions of labels in sample bags are given. We treat this problem as an instance of input training sample uncertainty modeled with KDMs. To ensure reproducibility, the implementation details of the experiments, along with the location of the source code, are provided in Appendix 2.

4.1 Bidirectional classification and generation with kernel density matrices

Figure 1 shows the task setup and models explored in this subsection. The upper part shows a classifier that combines a deep encoder, $\Phi_{\theta} : \mathbb{X} \rightarrow \mathbb{R}^n$, and a KDM inference module. The parameters of the model correspond to the joint KDM $\rho_{\mathbf{x}',\mathbf{y}'} = (\mathbf{C}_{\mathbf{x}'\mathbf{y}'}, \mathbf{p}_{\mathbf{x}'\mathbf{y}'}, k_{\text{rbf},\sigma} \otimes k_{\text{cos}})$, which represents the joint probability of the training input and output samples, and the parameters of the encoder, θ . These parameters, together with the parameters of the encoder, are learned using an extended version of Algorithm 3 by minimizing the loss given by Eq. 14.

The lower part of Fig. 1 shows a conditional generative model that exploits the reversibility of the KDM inference

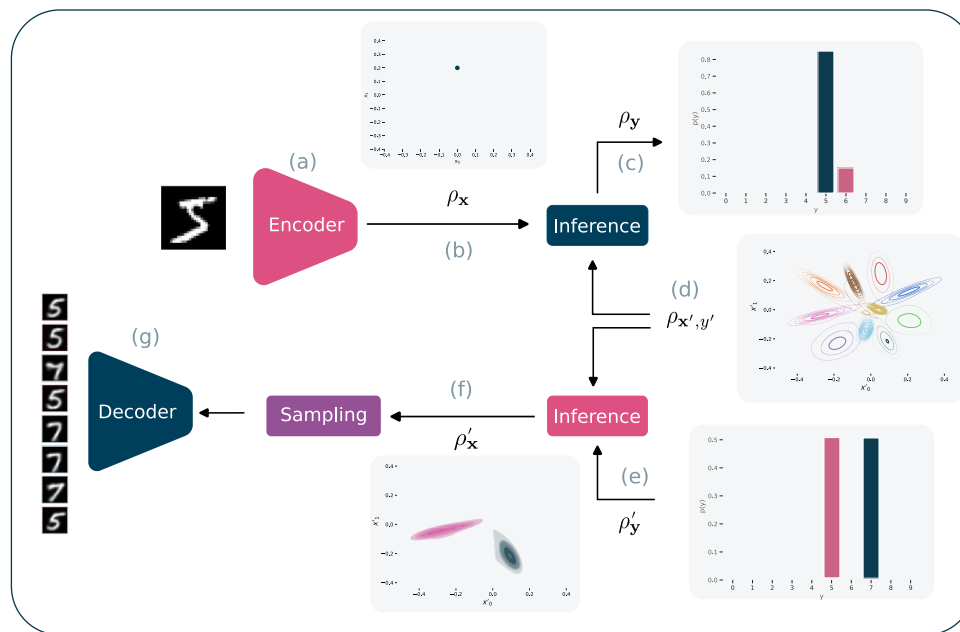


Fig. 1 classification and generation with KDMs. The top part represents a predictive model that uses an encoder (a) to map input samples into a latent space; the output of the encoder is represented as a KDM ρ_x (b) with one component, which is used to infer an output probability distribution of labels, represented by a KDM ρ_y (c), using Eq. 12; the classifier model has as a parameter a joint distribution of inputs and outputs, represented by a KDM $\rho_{x',y'}$ (d), which is learned with Algo-

rithm 3. The joint probability can be used to do conditional generation, as depicted in the bottom of the diagram. In this case, the input is a distribution of labels represented by the KDM ρ'_y (e), which along with $\rho_{x',y'}$ (d) is used to infer a predicted KDM ρ'_x using Eq. 12; we sample (f) from this KDM to generate input samples in the latent space which are decoded (g) to the original input space

process (Eq. 12). Thanks to the symmetry of $\rho_{x',y'}$, it is possible to use it to infer a probability distribution of inputs, ρ_x , given a probability distribution of outputs ρ_y . From this KDM, we can draw samples (Subsect. 3.6) in the latent space, which can be mapped to the input space using a decoder $\Psi_{\theta'} : \mathbb{R}^n \rightarrow \mathbb{X}$. To improve the quality of the generated samples, the joint KDM $\rho_{x',y'}$ is fine-tuned using maximum likelihood learning (see Subsect. 3.5) while keeping the parameters θ of the encoder fixed. The parameters θ' of the decoder are independently learned by training a conventional auto-encoder composed of Φ_θ and $\Psi_{\theta'}$, with the encoder parameters θ fixed.

Table 1 shows the performance of the model on three benchmark datasets: MNIST, Fashion-MNIST, and CIFAR-10. All models use a similar encoder architecture: a convolutional neural network (3 convolutional layers for MNIST and Fashion-MNIST, and 6 convolutional layers for CIFAR-10). As a baseline, we have coupled the encoder with a dense layer with approximately the same number of parameters as the KDM inference module (Encoder+Dense), so that we can compare KDM with a counterpart model of similar complexity. We also report the classification performance of the model before (KDM) and after performing the maximum likelihood fine-tuning (ML-KDM) on the joint KDM $\rho_{x',y'}$ for improving generation. Hyperparameter tuning of the models was

performed using cross-validation on the training partition for each dataset. Ten experiments were run with the best hyperparameters, and the mean accuracy is reported along with the 99% confidence interval of a t-test. The results show that the performance of the KDM-based model is on par with a deep model of comparable complexity.

The main advantage of the KDM-based model is the probabilistic modeling of the interaction between inputs and outputs, which is exploited by the conditional generative model. It uses the parameters learned by the ML-KDM to conditionally generate samples from the inferred probability distribution of the inputs. Some examples of conditionally generated samples for the three datasets are shown in Fig. 2.

Table 1 Classification accuracy comparison of the models on the three datasets: KDM classification model (KDM), KDM model fine-tuned for generation with maximum likelihood learning (ML-KDM), a baseline model with the same encoder coupled with a dense layer

Dataset	KDM	ML-KDM	Enc.+Dense
Mnist	0.993 ± 0.001	0.993 ± 0.001	0.992 ± 0.001
FMnist	0.916 ± 0.004	0.895 ± 0.004	0.907 ± 0.005
Cifar-10	0.811 ± 0.005	0.776 ± 0.012	0.810 ± 0.006

Observe that the classification performance ML-KDM slightly degrades with respect to KDM, since a more faithful modeling of the input space distribution impacts the discriminative performance

Fig. 2 Conditional image generation from Mnist, Fashion-Mnist, and Cifar-10 using the KDM conditional generative model, each row corresponds to a different class

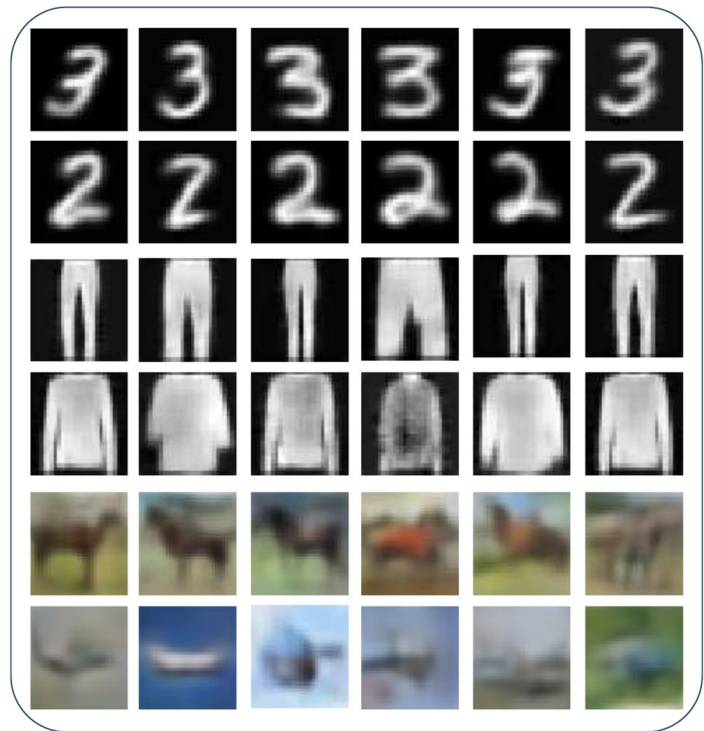


Table 2 Performance evaluation for the learning with label proportions tasks

Dataset, sampling	Method	8	32	128	512
Adult, $[0, \frac{1}{2}]$	LMMCM	0.8728 ± 0.0034	0.8693 ± 0.0084	0.8669 ± 0.0073	0.8674 ± 0.0072
	KDM	0.8797 ± 0.0054	0.8786 ± 0.0048	0.8757 ± 0.0056	0.8697 ± 0.0038
Adult, $[\frac{1}{2}, 1]$	LMMCM	0.8584 ± 0.0295	0.8644 ± 0.00937	0.8601 ± 0.00811	0.8500 ± 0.0335
	KDM	0.8810 ± 0.0083	0.8702 ± 0.0121	0.8666 ± 0.0081	0.8601 ± 0.0073
MAGIC, $[0, \frac{1}{2}]$	LMMCM	0.8909 ± 0.0013	0.8799 ± 0.0203	0.8753 ± 0.0283	0.8734 ± 0.0165
	KDM	0.8957 ± 0.00132	0.8766 ± 0.0083	0.8709 ± 0.0063	0.8578 ± 0.0043
MAGIC, $[\frac{1}{2}, 1]$	LMMCM	0.8911 ± 0.0149	0.8790 ± 0.0164	0.8684 ± 0.0083	0.8567 ± 0.0165
	KDM	0.8957 ± 0.0093	0.8793 ± 0.0121	0.8709 ± 0.0058	0.8578 ± 0.045

Column headers indicate bag size. Values correspond to AUC plus a t-test 99% confidence interval

4.2 Classification with label proportions

Learning from label proportions (LLP) is a type of weakly supervised problem (Scott and Zhang 2020; Zhang et al 2022). In this scenario, instead of having explicit class labels for each training instance, we only have class proportions for subsets of training instances, called bags. The goal of the learner is to build a classifier that accurately labels individual instances. A bag of instances with label proportions can be thought of as a training sample that has uncertainty about the inputs and outputs, as discussed in Section 3.5. This uncertainty can be represented by KDMs as shown in Fig. 3. During training, the model receives as input bags of instances $\mathbf{X}^{(i)} = (\mathbf{x}^{(i)j})_{j=1 \dots m_i}$. The training dataset corresponds to a set of pairs $\mathbf{D} = (\mathbf{X}^{(i)}, \mathbf{y}^{(i)})_{i=1 \dots \ell}$, where each

$\mathbf{y}^{(i)}$ is a vector representing the label proportions of the i -th bag. Each input bag is represented by a KDM with m_i components. The Algorithm 3 can be adapted to handle this bag representation by changing line 4, the resulting algorithm is shown in (Algorithm 4). The algorithm learns a joint KDM $\rho_{\mathbf{x}', \mathbf{y}'}$. During prediction, the model receives individual samples $\mathbf{x}^{(*)}$, which can be seen as bags with only one element. They are represented by a KDM $\rho_{\mathbf{x}}^{(*)} = ((\mathbf{x}^{(*)}), \{1\}, k_{\mathbb{X}})$ with a unique component and uses Eq. 12, along with $\rho_{\mathbf{x}', \mathbf{y}'}$, to infer an output KDM $\rho_{\mathbf{y}}$.

We compared this approach with a state-of-the-art LLP method based on a mutual contamination framework (LMMCM) (Scott and Zhang 2020). We followed the same experimental setup as the one used by Scott and Zhang (2020), where the authors built a set of bags of different sizes with the

Algorithm 4 Discriminative training with bags of instances.

input: training dataset $\mathbf{D} = \{(\mathbf{X}^{(i)}, \mathbf{y}^{(i)})\}_{i=1 \dots \ell}$ with $\mathbf{X}^{(i)} = (\mathbf{x}^{(i)j})_{j=1 \dots m_i}$, kernels $k_{\mathbb{X}, \sigma}$ and $k_{\mathbb{Y}}$ number of components m' , loss function $\mathcal{L}$

- 1: Initialize $\mathbf{C}_{\mathbf{x}'\mathbf{y}'} \leftarrow ((\mathbf{x}'^{(i)}, \mathbf{y}'^{(i)}))_{i=1 \dots m'}$ with a random sample from $\mathbf{D}$
- 2: $\mathbf{p}_{\mathbf{x}'\mathbf{y}'} \leftarrow (\frac{1}{m'})_{i=1 \dots m'}$
- 3: $\rho_{\mathbf{x}'\mathbf{y}'} \leftarrow (\mathbf{C}_{\mathbf{x}'\mathbf{y}'}, \mathbf{p}_{\mathbf{x}'\mathbf{y}'}, k_{\mathbb{X}, \sigma} \otimes k_{\mathbb{Y}})$
- 4: $\rho_{\mathbf{x}}^{(i)} \leftarrow (\mathbf{X}^{(i)}, (\frac{1}{m_i}, \dots, \frac{1}{m_i}), k_{\mathbb{X}})$ for all $i = 1 \dots \ell$
- 5: **repeat**
- 6: Calculate $\rho_{\mathbf{y}}^{(i)}$ using Eq. 12 with parameters $\rho^{(i)}(\mathbf{x})$ and $\rho(\mathbf{x}', \mathbf{y}')$ for all $i = 1 \dots \ell$
- 7: Perform a step of a gradient-based method to optimize: $\min_{\mathbf{C}_{\mathbf{x}'\mathbf{y}'}, \mathbf{p}_{\mathbf{x}'\mathbf{y}'}, \sigma} \sum_{i=1}^n \mathcal{L}(\rho_{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)})$
- 8: **until** stop criteria are met
- 9: **return** $\rho_{\mathbf{x}'\mathbf{y}'} = (\mathbf{C}_{\mathbf{x}'\mathbf{y}'}, \mathbf{p}_{\mathbf{x}'\mathbf{y}'}, k_{\mathbb{X}} \otimes k_{\mathbb{Y}})$

corresponding label proportions from a conventional classification dataset. The elements of the bags are sampled in such a way that the label proportions are iid uniform on $[0, \frac{1}{2}]$ and on $[\frac{1}{2}, 1]$. The base datasets are Adult and MAGIC Gamma Ray Telescope from the UCI repository.⁴

Table 2 shows the evaluation of the LMMCM and the KDM methods using the area under the ROC curve (AUC) metric. The results are reported for different bag sizes, the two data sets, and the $[0, \frac{1}{2}]$ and $[\frac{1}{2}, 1]$ sampling schemes. The results show that KDM has a competitive performance

⁴ <http://archive.ics.uci.edu/ml>

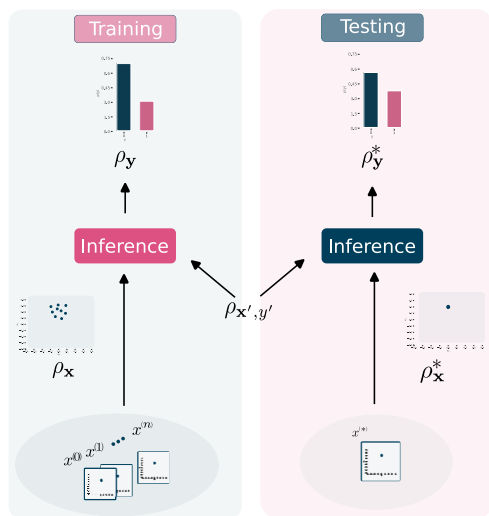


Fig. 3 KDM model for classification with label proportions. During training, the model receives as input bags of instances $\mathbf{X}^{(i)} = (\mathbf{x}^{(i)j})_{j=1 \dots m_i}$. The training dataset corresponds to a set of pairs $\mathbf{D} = (\mathbf{X}^{(i)}, \mathbf{y}^{(i)})_{i=1 \dots \ell}$, where each $\mathbf{y}^{(i)}$ is a vector representing the label proportions of the i -th bag. Each input is represented by a KDM with m_i components. The algorithm learns a joint KDM $\rho_{\mathbf{x}'\mathbf{y}'}$. During prediction, the model receives individual samples, $\mathbf{x}^{(*)}$, (equivalent to bags with only one element). The algorithm outputs a KDM $\rho_{\mathbf{y}}$

equal to that of a state-of-the-art method explicitly designed for the LLP task. The good performance of the KDM method is a result of its ability to naturally represent the uncertainty of the training samples.

5 Conclusions

The KDM framework presents a simple yet effective mechanism for representing joint probability distributions of both continuous and discrete random variables, providing a versatile tool for various machine learning tasks. The exploratory experiments shed light on the potential of this framework and show how it can be applied to diverse tasks such as conditional generation and weakly supervised learning. Quantum density matrices have long been used as the foundation of quantum mechanics, and their ability to blend linear algebra and probability is very attractive as a tool for machine learning. This work illustrates how such a formalism can be seamlessly and efficiently integrated with probabilistic deep learning models, opening up new avenues of research on the intersection of these fields.

Appendix 1: Proofs

Proposition 1 Let $\rho_{\mathbf{x}} = (\mathbf{C}, \mathbf{p}, k_{\cos})$ be a KDM over $\mathbb{R}^n$; let $\mathbb{X} = \{\mathbf{b}^{(1)}, \dots, \mathbf{b}^{(n)}\} \subset \mathbb{R}^n$ be an orthogonal basis of $\mathbb{R}^n$, then $\{f_{\rho}(\mathbf{b}^{(i)})\}_{i=1 \dots n}$ is a categorical probability distribution for the random variable $\mathbf{x} \in \mathbb{X}$.

Proof Let $\mathbf{C} = \{\mathbf{x}^{(j)}\}_{j=1 \dots m}$, then $f_{\rho_{\mathbf{x}}}(\mathbf{b}^{(i)}) = \sum_{j=1}^m p_j \frac{\langle \mathbf{b}^{(i)}, \mathbf{x}^{(j)} \rangle^2}{\langle \mathbf{x}^{(j)}, \mathbf{x}^{(j)} \rangle} \geq 0$.

$$\begin{aligned}
 \sum_{i=1}^n f_{\rho_{\mathbf{x}}}(\mathbf{b}^{(i)}) &= \sum_{i=1}^n \sum_{j=1}^m p_j \frac{\langle \mathbf{b}^{(i)}, \mathbf{x}^{(j)} \rangle^2}{\langle \mathbf{x}^{(j)}, \mathbf{x}^{(j)} \rangle} \\
 &= \sum_{j=1}^m p_j \sum_{i=1}^n \frac{\langle \mathbf{b}^{(i)}, \mathbf{x}^{(j)} \rangle^2}{\langle \mathbf{x}^{(j)}, \mathbf{x}^{(j)} \rangle} \\
 &= \sum_{j=1}^m p_j \|\frac{\mathbf{x}^{(j)}}{\langle \mathbf{x}^{(j)}, \mathbf{x}^{(j)} \rangle}\|^2 \quad (\mathbb{X} \text{ is a basis of } \mathbb{R}^n) \\
 &= \sum_{j=1}^m p_j \\
 &= 1
 \end{aligned}$$

□

Proposition 3 Let $\mathbb{X} = \{\mathbf{e}^{(1)}, \dots, \mathbf{e}^{(n)}\}$ be the canonical basis of $\mathbb{R}^n$; let $\mathbf{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(\ell)}\} \subseteq \mathbb{R}^n$ be a set of iid

samples drawn from $\mathbb{X}$ with categorical probability distribution $\mathbf{p} = (p_1, \dots, p_n)$; let $\rho_{\mathbf{x}}^\ell = (\mathbf{D}, \mathbf{p} = (\frac{1}{\ell}, \dots, \frac{1}{\ell}), k_{\cos})$ be a KDM; and let $\hat{f}_{\rho_{\mathbf{x}}^\ell}(\mathbf{x})$ be defined as in Eq. 5. Then, $\hat{f}_{\rho_{\mathbf{x}}^\ell}(\mathbf{e}^{(i)}) \rightarrow p_i$ as $\ell \rightarrow \infty$.

Proof

$$\begin{aligned} f_{\rho_{\mathbf{x}}}(\mathbf{e}^{(i)}) &= \sum_{j=1}^{\ell} \frac{1}{\ell} k_{\cos}^2(\mathbf{x}^{(j)}, \mathbf{e}^{(i)}) \\ &= \sum_{j=1}^{\ell} \frac{1}{\ell} \langle \mathbf{x}^{(j)}, \mathbf{e}^{(i)} \rangle^2 \\ &= \sum_{j=1}^{\ell} \frac{1}{\ell} 1(\mathbf{x}^{(j)} = \mathbf{e}^{(i)}) \end{aligned}$$

Because of the large numbers law:

$$\begin{aligned} \lim_{\ell \rightarrow \infty} f_{\rho_{\mathbf{x}}}(\mathbf{e}^{(i)}) &= \lim_{\ell \rightarrow \infty} \sum_{j=1}^{\ell} \frac{1}{\ell} 1(\mathbf{x}^{(j)} = \mathbf{e}^{(i)}) \\ &= E[1(\mathbf{x}^{(j)} = \mathbf{e}^{(i)})] \\ &= p_i \end{aligned} \quad (\text{A1})$$

□

Proposition 4 Let $\mathbf{x}'$ and $\mathbf{y}'$ be random variables with a joint probability distribution represented by a KDM $\rho_{\mathbf{x}', \mathbf{y}'}$ (Eq. 10), and let $\rho_{\mathbf{x}}$ be the KDM defined by Eq. 9), then the KDM $\rho_{\mathbf{y}}$ (Eqs. 11 and 12) represents a predicted probability distribution with PDF:

$$\hat{f}_{\rho_{\mathbf{y}}}(\mathbf{y}) = \sum_{\ell=1}^m p_{\ell} \hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{y} | \mathbf{x}^{(\ell)})$$

where

- $\hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{y} | \mathbf{x}^{(\ell)}) = \frac{\hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{x}^{(\ell)}, \mathbf{y})}{\int \hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{x}^{(\ell)}, \mathbf{y}) d\mathbf{y}}$ for $k_{\mathbb{Y}} = k_{\text{rbf}}$
- $\hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{y} | \mathbf{x}^{(\ell)}) = \frac{\hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{x}^{(\ell)}, \mathbf{y})}{\sum_{i=1}^n \hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{x}^{(\ell)}, \mathbf{e}^{(i)})}$ for $k_{\mathbb{Y}} = k_{\cos}$

Proof Step 1. Show that for both $k_{\mathbb{Y}} = k_{\text{rbf}}$ and $k_{\mathbb{Y}} = k_{\cos}$

$$\hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{y} | \mathbf{x}^{(\ell)}) = \frac{\hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{x}^{(\ell)}, \mathbf{y})}{\mathcal{M}_{k_{\mathbb{X}}} \sum_{j=1}^{m'} p'_j k_{\mathbb{X}}^2(\mathbf{x}^{(\ell)}, \mathbf{x}'^{(j)})} \quad (\text{A2})$$

The case $k_{\mathbb{Y}} = k_{\text{rbf}}$ follows from Eq. 8. For the case $k_{\mathbb{Y}} = k_{\cos}$, we have

$$\sum_{i=1}^n \hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{x}^{(\ell)}, \mathbf{e}^{(i)})$$

$$\begin{aligned} &= \sum_{i=1}^n \sum_{j=1}^{m'} p'_j \mathcal{M}_{k_{\mathbb{X}}} \mathcal{M}_{k_{\cos}} k_{\mathbb{X}}^2(\mathbf{x}^{(\ell)}, \mathbf{x}'^{(j)}) k_{\cos}^2(\mathbf{e}^{(i)}, \mathbf{y}'^{(j)}) \\ &= \mathcal{M}_{k_{\mathbb{X}}} \sum_{j=1}^{m'} p'_j k_{\mathbb{X}}^2(\mathbf{x}^{(\ell)}, \mathbf{x}'^{(j)}) \sum_{i=1}^n k_{\cos}^2(\mathbf{e}^{(i)}, \mathbf{y}'^{(j)}) \\ &= \mathcal{M}_{k_{\mathbb{X}}} \sum_{j=1}^{m'} p'_j k_{\mathbb{X}}^2(\mathbf{x}^{(\ell)}, \mathbf{x}'^{(j)}) \end{aligned}$$

Step 2. Use the derivation above to prove the main result:

$$\begin{aligned} &\hat{f}_{\rho_{\mathbf{y}}}(\mathbf{y}) \\ &= \mathcal{M}_{k_{\mathbb{Y}}} \sum_{i=1}^{m'} p''_i k_{\mathbb{Y}}^2(\mathbf{y}, \mathbf{y}'^{(i)}) \quad (\text{use Eq. 4}) \\ &= \mathcal{M}_{k_{\mathbb{Y}}} \sum_{\ell=1}^m \sum_{i=1}^{m'} \frac{p_{\ell} p'_i k_{\mathbb{X}}^2(\mathbf{x}^{(\ell)}, \mathbf{x}'^{(i)})}{\sum_{j=1}^{m'} p'_j k_{\mathbb{X}}^2(\mathbf{x}^{(\ell)}, \mathbf{x}'^{(j)})} k_{\mathbb{Y}}^2(\mathbf{y}, \mathbf{y}'^{(i)}) \quad (\text{use Eq. 12}) \\ &= \sum_{\ell=1}^m p_{\ell} \frac{\mathcal{M}_{k_{\mathbb{X}}} \mathcal{M}_{k_{\mathbb{Y}}} \sum_{i=1}^{m'} p'_i k_{\mathbb{X}}^2(\mathbf{x}^{(\ell)}, \mathbf{x}'^{(i)}) k_{\mathbb{Y}}^2(\mathbf{y}, \mathbf{y}'^{(i)})}{\mathcal{M}_{k_{\mathbb{X}}} \sum_{j=1}^{m'} p'_j k_{\mathbb{X}}^2(\mathbf{x}^{(\ell)}, \mathbf{x}'^{(j)})} \\ &= \sum_{\ell=1}^m p_{\ell} \frac{\hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{x}^{(\ell)}, \mathbf{y})}{\mathcal{M}_{k_{\mathbb{X}}} \sum_{j=1}^{m'} p'_j k_{\mathbb{X}}^2(\mathbf{x}^{(\ell)}, \mathbf{x}'^{(j)})} \\ &= \sum_{\ell=1}^m p_{\ell} \hat{f}_{\rho_{\mathbf{x}', \mathbf{y}'}}(\mathbf{y} | \mathbf{x}^{(\ell)}) \quad (\text{use Eq. A2}) \end{aligned}$$

□

Appendix 2: Experimental setup

In this appendix, we provide a comprehensive description of our experimental setup to ensure reproducibility.

2.1: Hardware specifications

Our experiments were conducted using a server that incorporates a 64-core Intel Xeon Silver 4216 CPU 2.10 GHz processor, 128 GB of RAM, and two NVIDIA RTX A5000 GPUs.

2.2: Classification with kernel density matrices

Three different models were assessed in this experiment, as explained in Section 4.1: KDM classification model (KDM), KDM model fine-tuned for a generation with maximum likelihood learning (ML-KDM), and a baseline model with the same encoder coupled with a dense layer. The encoder model utilized for both the MNIST and Fashion-MNIST datasets was identical. It consisted of the following components:

- **Lambda Layer:** The initial layer converted each sample into a 32-float number and subtracted 0.5 from it.
- **Convolutional Layers:** Two convolutional layers were appended, each with 32 filters, 5-kernel size, same padding, and strides 1 and 2, respectively. Subsequently, two more convolutional layers were added, each with 64 filters, 5-kernel size, same padding, and strides 1 and 2, respectively. Finally, a convolutional layer with 128 filters, 7-kernel size, and a stride 1 was included. All convolutional layers employed the Gaussian Error Linear Unit (GELU) activation function.
- **Flattened Layer:** The output from the previous layer was flattened.
- **Dropout Layer:** A dropout layer with a dropout rate of 0.2 was introduced.
- **Dense Layer:** The subsequent dense layer's neuron encoding size was determined using a grid hyperparameter search in the range of 2^i , where $i \in \{1, \dots, 7\}$.

The decoder architecture consisted of the following components:

- **Reshape Layer:** The input to the decoder passed through a reshape layer.
- **Convolutional Layers:** Three convolutional layers were employed, each with 64 filters and kernel sizes of seven, five, and five, respectively. The strides for these layers were set to 1, 1, and 2, while the padding was configured as valid, same, and same, respectively.
- **Additional Convolutional Layers:** Subsequently, three more convolutional layers were utilized, each with 32 filters, 5-kernel size, and strides 1, 2, and 1, respectively. All these layers utilized the Gelu activation function and had padding set to "same."
- **Final Convolutional Layer:** The decoder concluded with a 1-filter convolutional layer employing same padding.

Similar to the encoder, all layers in the decoder employed the GELU activation function.

For Cifar-10 a different encoder was used for

- **Convolutional Layers:** The input tensor is passed through a series of convolutional layers. Each layer applies a $3 \times$

3 kernel to extract features from the input. The activation function used is the GELU. Padding is set to "same." Batch normalization is applied after each convolutional layer to normalize the activations.

- **Max Pooling:** After each pair of convolutional layers, max pooling is performed using a 2×2 pool size.
- **Encoding Layer:** The feature maps obtained from the previous layers are flattened into a vector representation using the Flatten layer.
- **Dropout:** A dropout layer with a dropout rate of 0.2 is added.
- **Hidden Layer:** A dense layer of variable size is introduced to further transform the encoded features. The size is search
- **Dropout:** Another dropout layer with a dropout rate of 0.2 is added after the hidden layer

The decoder architecture for Cifar-10 consisted of the following components:

- **Dense Layer:** The input tensor is passed through a dense layer that restores the tensor to its original size.
- **Reshape Layer:** The tensor is reshaped to match the spatial dimensions of the original input using the Reshape layer.
- **Convolutional Transpose Layers:** The reshaped tensor undergoes a series of convolutional transpose layers. Each layer applies a 3×3 kernel to upsample the feature maps. The activation function used is "gelu." Padding is set to "same" to maintain the spatial dimensions.
- **Batch Normalization:** Batch normalization is applied after each convolutional transpose layer to normalize the activations.
- **Upsampling:** UpSampling2D layers with a size of (2, 2) are used to increase the spatial dimensions of the feature maps.
- **Final Convolutional Transpose Layer:** The last convolutional transpose layer reconstructs the original number of channels in the input data. Padding is set to "same."
- **Activation Function:** The final output is passed through an activation function, sigmoid, to ensure the reconstructed data is within the appropriate range.

Table 3 Best parameters for the classification with kernel density matrix experiment

Dataset	KDM		KDM-ML		Baseline dense layer	
	Number of components	Encoded size	Number of components	Encoded size	Number of components	Encoded size
Mnist	512	128	512	128	8	8
Fashin-Mnist	256	128	256	128	4	16
Cifar-10	1024	32	1024	32	4	64

Table 4 Best hyper-parameters for the learning with label proportion experiment using KDM Model

Dataset, sample	Bag size	Num. components	Learning rate
Adult, $[0, \frac{1}{2}]$	8	32	0.005
Adult, $[0, \frac{1}{2}]$	32	16	0.001
Adult, $[0, \frac{1}{2}]$	128	512	0.001
Adult, $[0, \frac{1}{2}]$	512	64	0.005
Adult, $[\frac{1}{2}, 1]$	8	16	0.005
Adult, $[\frac{1}{2}, 1]$	32	64	0.001
Adult, $[\frac{1}{2}, 1]$	128	128	0.001
Adult, $[\frac{1}{2}, 1]$	512	64	0.005
MAGIC, $[0, \frac{1}{2}]$	8	256	0.005
MAGIC, $[0, \frac{1}{2}]$	32	128	0.005
MAGIC, $[0, \frac{1}{2}]$	128	128	0.001
MAGIC, $[0, \frac{1}{2}]$	512	256	0.001
MAGIC, $[\frac{1}{2}, 1]$	8	16	0.005
MAGIC, $[\frac{1}{2}, 1]$	32	128	0.005
MAGIC, $[\frac{1}{2}, 1]$	128	32	0.005
MAGIC, $[\frac{1}{2}, 1]$	512	128	0.001

The number of components in every dataset for the KDM classification model was searched in 2^i , where $i \in \{2, \dots, 10\}$. The same for ML-KQM. For the baseline model, the dense layer was searched using the same number of parameters that the KDM classification model generates. The best parameters can be found in Table 3.

The results reported in Table 1 were generated by running each algorithm in each dataset ten times and averaging their results.

2.3. Classification with label proportions

The experimental setup of this experiment is based on the work of Scott and Zhang (2020). The study utilizes two datasets, namely Adult ($T = 8192$) and MAGIC Gamma Ray Telescope ($T = 6144$), both obtained from the UCI repository.⁵ For each dataset, the authors created datasets with bags of instances with different proportions of LPs. Each dataset has its distribution of label proportions (LPs), and a bag size parameter, denoted as “ n .” The total number of training instances, denoted as “ T ,” is fixed for each dataset, resulting in T/n bags. The LPs for the experiments are independently and uniformly distributed within the ranges $[0, 1/2]$ and $[1/2, 1]$. The bag sizes considered are $n = \{8, 32, 128, \text{ and } 512\}$. Consequently, there are a total of 16 experimental configurations (2 LP distributions $\times 2$ datasets $\times 4$ bag sizes). For each configuration, 5 different pairs of training

and test sets were created. Results are reported as the average of the AUC over the five test sets of each configuration. The numerical features in both datasets are standardized to have a mean of 0 and a variance of 1, while the categorical features are encoded using one-hot encoding. Hyperparameters are found using a validation subset (10%) of each training set. The best hyperparameters for each configuration are reported in Table 4

2.4. Code

The code repository at <https://github.com/fagonzalezo/kdm> provides detailed instructions to reproduce the results presented in this manuscript.

For conducting experiments involving learning with label proportions, as mentioned above, we have relied on the work of Scott and Zhang (2020). The authors made the code available at this GitHub repository <https://github.com/Z-Jianxin/Learning-from-Label-Proportions-A-Mutual-Contamination-Framework>.

Author contributions F.G. and R.R. wrote the main manuscript text. A.G. ran experiments, prepared tables and figures. All authors reviewed the manuscript.

Funding Open Access funding provided by Colombia Consortium.

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Bengio Y, Larochelle H, Vincent P (2005) Non-local manifold parzen windows. In: Weiss Y, Schölkopf B, Platt J (eds) Advances in neural information processing systems, vol 18. MIT Press. https://proceedings.neurips.cc/paper_files/paper/2005/file/17eb7ecc4c38e4705361cccd903ad8c6-Paper.pdf

⁵ <http://archive.ics.uci.edu/ml>.

- Bishop CM (1994) Mixture density networks. Technical report, Aston University, https://publications.aston.ac.uk/id/eprint/373/1/NCRG_94_004.pdf
- Cappé O, Moulines E (2009) On-line expectation-maximization algorithm for latent data models. *J Royal Stat Soc Ser B Stat Methodol* 71(3):593–613. <https://doi.org/10.1111/J.1467-9868.2009.00698.X>
- Cerezo M, Verdon G, Huang HY et al (2023) Challenges and opportunities in quantum machine learning. *Nat Comput Sci* 2(9):567–576. <https://doi.org/10.1038/s43588-022-00311-3>
- Cho Y, Saul LK (2009) Kernel methods for deep learning. *Adv Neural Inf Process Syst* 22
- Damianou AC, Lawrence ND (2013) Deep Gaussian processes. *J Mach Learn Res* 31:207–215
- Dempster AP, Laird NM, Rubin DB (1977) Maximum likelihood from incomplete data via the EM algorithm. *J Royal Stat Soc Ser B Stat Methodol* 39(1):1–22. <https://doi.org/10.1111/J.2517-6161.1977.TB01600.X>
- Dilokthanakul N, Mediano PAM, Garnelo M et al (2016) Deep unsupervised clustering with Gaussian mixture variational autoencoders. *arXiv preprint arXiv:1611.02648*
- Figurnov M, Mohamed S, Mnih A (2018) Implicit reparameterization gradients. *Adv Neural Inf Process Syst* 2018-December:441–452. [arXiv:1805.08498v4](https://arxiv.org/abs/1805.08498v4)
- Giuntini R, Holik F, Park DK et al (2023) Quantum-inspired algorithm for direct multi-class classification. *Appl Soft Comput* 134. <https://doi.org/10.1016/j.asoc.2022.109956>
- Goan E, Fookes C (2020) Bayesian neural networks: an introduction and survey. *Lect Notes Math* 2259:45–87. https://doi.org/10.1007/978-3-030-42553-1_3/FIGURES/8
- González FA, Gallego A, Toledo-Cortés S et al (2022) Learning with density matrices and random features. *Quantum Mach Intell* 4(2). <https://doi.org/10.1007/s42484-022-00079-9>
- Goodfellow I, Pouget-Abadie J, Mirza M et al (2020) Generative adversarial networks. *Commun ACM* 63(11):139–144. <https://doi.org/10.1145/3422622>
- Greff K, van Steenkiste S, Schmidhuber J (2017) Neural expectation maximization. *Adv Neural Inf Process Syst* 30
- Jang E, Gu S, Poole B (2017) Categorical reparameterization with Gumbel-Softmax. 5th international conference on learning representations, ICLR 2017 - Conference Track Proceedings
- Jiang Z, Zheng Y, Tan H et al (2017) Variational deep embedding: an unsupervised and generative approach to clustering. In: proceedings of the 26th international joint conference on artificial intelligence (IJCAI), pp 1965–1972. <https://www.ijcai.org/proceedings/2017/0273.pdf>
- Kingma DP, Welling M (2013) Auto-encoding variational Bayes. 2nd International conference on learning representations, ICLR 2014 - Conference Track Proceedings. [arXiv:1312.6114v11](https://arxiv.org/abs/1312.6114v11)
- Li Q, Gkoumas D, Lioma C et al (2021) Quantum-inspired multimodal fusion for video sentiment analysis. *Inf Fusion* 65:58–71. <https://doi.org/10.1016/j.inffus.2020.08.006>
- Mukherjee S, Asnani H, Lin E et al (2019) ClusterGAN: latent space clustering in generative adversarial networks. In: Proceedings of the 33rd AAAI conference on artificial intelligence (AAAI), pp 4610–4617. <https://arxiv.org/pdf/1809.03627.pdf>
- Nadaraya EA (1964) On estimating regression. *Theory Probab Appl* 9(1):141–142. <https://doi.org/10.1137/1109020>
- Nielsen MA, Chuang IL (2012) Quantum computation and quantum information. Cambridge University Press. <https://doi.org/10.1017/CBO9780511976667>
- Ober SW, Rasmussen CE, van der Wilk M (2021) The promises and pitfalls of deep kernel learning. In: de Campos C, Maathuis MH (eds) Proceedings of the 37th conference on uncertainty in artificial intelligence, Proceedings of machine learning research, vol 161. PMLR, pp 1206–1216. <https://proceedings.mlr.press/v161/ober21a.html>
- van den Oord A, Dieleman S, Zen H et al (2016) WaveNet: a generative model for raw audio. In: Proceedings of the 9th ISCA speech synthesis workshop, p 125. <https://research.google/pubs/pub45611/>
- Papamakarios G, Nalisnick E, Rezende DJ et al (2021) Normalizing flows for probabilistic modeling and inference. *J Mach Learn Res* 22:1–64. <http://jmlr.org/papers/v22/19-1028.html>
- Parzen E (1962) On estimation of a probability density function and mode. *Ann Math Stat* 33(3):1065–1072. <https://www.jstor.org/stable/2237880>
- Rahimi A, Recht B (2007) Random features for large-scale kernel machines. *Adv Neural Inf Process Syst* 20
- Rezende DJ, Mohamed S (2015) Variational inference with normalizing flows. 32nd international conference on machine learning, ICML 2015 2:1530–1538
- Rosenblatt M (1956) Remarks on some nonparametric estimates of a density function. *Ann Math Stat* 27(3):832–837. <https://www.jstor.org/stable/2237390>
- Salimans T, Karpathy A, Chen X et al (2017) PixelCNN++: improving the PixelCNN with discretized logistic mixture likelihood and other modifications. In: Proceedings of the 5th international conference on learning representations (ICLR). <https://openreview.net/pdf?id=BJrFC6ceg>
- Schuld M, Petruccione F (2018) Supervised learning with quantum computers. Quantum Sci Techn, Springer International Publishing, Cham. <https://doi.org/10.1007/978-3-319-96424-9>
- Scott C, Zhang J (2020) Learning from label proportions: a mutual contamination framework. *Adv Neural Inf Process Syst*
- Sergioli G, Santucci E, Didaci L et al (2018) A quantum-inspired version of the nearest mean classifier. *Soft Comput* 22(3):691–705. <https://doi.org/10.1007/s00500-016-2478-2>
- Shimizu E, Fukumizu K, Sejdinovic D (2024) Neural-kernel conditional mean embeddings. <https://proceedings.mlr.press/v235/shimizu24a.html>
- Srinivasan S, Downey C, Boots B (2018) Learning and inference in Hilbert space with quantum graphical models. *Adv Neural Inf Process Syst* 2018-December:10338–10347. [arXiv:1810.12369](https://arxiv.org/abs/1810.12369)
- Tiwari P, Melucci M (2019) Towards a quantum-inspired binary classifier. *IEEE Access* 7:42354–42372. <https://doi.org/10.1109/ACCESS.2019.2904624>
- Vincent P, Bengio Y (2002) Manifold Parzen windows. In: Becker S, Thrun S, Obermayer K (eds) Advances in Neural Information Processing Systems, vol 15. MIT Press, https://proceedings.neurips.cc/paper_files/paper/2002/file/2d969e2cee8cfa07ce7ca0bb13c7a36d-Paper.pdf
- Wang T, Zhang L, Hu W (2021) Bridging deep and multiple kernel learning: a review. *Inf Fusion* 67:3–13. <https://doi.org/10.1016/J.INFFUS.2020.10.002>
- Wasserman L, Berlin S, New H et al (2006) All of non-parametric statistics. https://books.google.com/books?hl=en&lr=&id=MRFlzQfRg7UC&oi=fnd&pg=PA2&dq=all+of+nonparametric+statistics&ots=SQSYub60CB&sig=_56li4lv7SDSVmGJlMK_uXm-gfE
- Watson GS (1964) Smooth regression analysis. *Sankhyā: The Indian Journal of Statistics. Ser A* (1961-2002) 26(4):359–372. <http://www.jstor.org/stable/25049340>
- Wolf L (2006) Learning using the born rule. Technical Report, Massachusetts institute of technology computer science and artificial intelligence laboratory
- Zhang J, Wang Y, Scott C (2022) Learning from label proportions by learning with label noise. *Adv Neural Inf Process Syst* 35:26933–26942. [arXiv:2203.02496](https://arxiv.org/abs/2203.02496)
- Zhang P, Peng J, Riedel N (2005) Finite sample error bound for Parzen windows. *Proc National Confer Artif Intell* 2:925–930

- Zong B, Song Q, Min MR et al (2018) Deep autoencoding gaussian mixture model for unsupervised anomaly detection. In: Proceedings of the 6th international conference on learning representations (ICLR). <https://openreview.net/pdf?id=BJJLHbb0->
- Zwiebach B (2022) Mastering quantum mechanics: essentials, theory, and applications. The MIT Press, https://books.google.com/books?hl=en&lr=&id=O_E3EAAQBAJ&oi=fnd&pg=PR15&dq=mastering+quantum+mechanics&ots=aqzk_hbVd2&sig=bmH7bgSW84NcKlINPmQQfqYmnV0

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.